

OpenContrails — Détection automatique des traînées de condensation dans les images satellites

Septembre 2026

Contents

1. Le problème	1
2. OpenContrails et structure des données	2
3. Préparation des données	2
4. La baseline U-Net	3
5. Ce qui existe déjà (littérature et Kaggle)	4
6. Pistes de recherche à discuter	4
7. Plan de travail en 7 phases	4
8. Décisions à prendre ensemble	5
Liens et ressources — récapitulatif	6

Document de cadrage à discuter ensemble

Ce document résume le problème, les données, l'approche de modélisation envisagée, ce qui existe déjà dans la littérature et sur Kaggle, ainsi que les pistes de recherche et les décisions qu'il reste à prendre à deux avant de se lancer sérieusement dans l'implémentation.

1. Le problème

On veut développer un système de vision par ordinateur capable de localiser automatiquement les **contrails** (traînées de condensation) dans des observations satellites. Le problème est formulé comme une **segmentation sémantique binaire** : pour chaque pixel d'une image satellite, le modèle doit décider s'il appartient ou non à une contrail.

Pourquoi c'est important. Les contrails persistantes contribuent à l'impact climatique de l'aviation. Un système automatique de détection permet de mieux mesurer les contrails observées et d'évaluer des stratégies de prévision ou d'évitement.

Le phénomène physique. Une contrail est un nuage linéaire de cristaux de glace pouvant se former derrière un avion lorsque les gaz d'échappement rencontrent, en altitude, des conditions atmosphériques suffisamment froides et humides. Certaines traînées disparaissent rapidement ; d'autres persistent, s'élargissent et peuvent évoluer vers des structures ressemblant à des cirrus.

Pourquoi c'est difficile : - Une contrail n'est pas toujours une ligne blanche nette : elle peut se diffuser et se mélanger aux cirrus. - Une seule image peut être ambiguë ; l'évolution dans le temps apporte une information discriminante. - La classe positive

occupe souvent une petite fraction de l'image : le problème est fortement déséquilibré. - Même les annotateurs humains peuvent être en désaccord sur certains pixels ou certaines structures.

2. OpenContrails et structure des données

Le jeu de données central est [OpenContrails](#), publié par Google Research à partir des observations de l'instrument **ABI** du satellite météorologique géostationnaire **GOES-16**.

- **GOES-16** est géostationnaire : il observe continuellement une même région depuis une position apparente fixe par rapport à la Terre, ce qui permet d'étudier l'évolution temporelle des nuages et des contrails.
- **ABI (Advanced Baseline Imager)** mesure plusieurs bandes spectrales, notamment dans l'infrarouge. Dans OpenContrails, les fichiers `band_08.npy` à `band_16.npy` correspondent à ces différentes bandes, converties en températures de brillance.
- Le dataset fournit **8 instants par exemple** : 4 avant la frame annotée, la frame cible, puis 3 après, à intervalles de 10 minutes. Chaque exemple ne possède qu'une seule frame annotée comme cible.

Structure typique d'un record :

```
record_id/  
  band_08.npy ... band_16.npy      # H × W × T (T = dimension temporelle)  
  human_individual_masks.npy      # annotations individuelles (train uniquement)  
  human_pixel_masks.npy          # vérité terrain agrégée (masque majoritaire)
```

`human_pixel_masks` : un pixel est positif lorsque plus de la moitié des annotateurs l'ont marqué comme contrail. Les consignes de labellisation de la compétition imposent qu'une contrail soit suffisamment longue, apparaisse soudainement ou entre par le bord de l'image, et soit visible dans au moins deux frames — ce qui explique pourquoi le contexte temporel est important.

Volumétrie. La version Kaggle du dataset représente environ **450,91 Go** et plus de 244 000 fichiers — à ne surtout pas télécharger intégralement au début. On travaille sur un sous-échantillon ou directement dans un environnement Kaggle/Cloud.

Liens : - Article de référence : [Contrail Detection on GOES-16 ABI with the OpenContrails Dataset](#) - Page projet Google : [Google Project Contrails](#) - Datasets publics Google Research : sites.research.google/gr/contrails/public-datasets - Compétition Kaggle : [google-research-identify-contrails-reduce-global-warming](#) - Description des données Kaggle : [.../data](#)

3. Préparation des données

Avant tout entraînement, l'objectif de la Phase 1 est de : 1. Récupérer un **mini-échantillon** (1 à 5 records) plutôt que le dataset complet — voir le [mini-sample OpenContrails sur Kaggle](#). 2. Charger un record et vérifier ses dimensions réelles plutôt que de les supposer. 3. Construire une représentation visualisable (par ex. composite ASH/RGB) des bandes infrarouges. 4. Inspecter les annotations individuelles et mesurer le désaccord entre annotateurs. 5. Garder les données **hors du dépôt Git**,

dans un dossier local ignoré par .gitignore (les fichiers .npy/.nc/.h5/checkpoints ne doivent jamais être versionnés).

Pipeline conceptuel proposé :

GOES-16 / ABI → Prétraitement (ASH/RGB, tenseurs) → Réseau IA (U-Net / DeepLab / etc.) → Sortie (masque de contrails)

Formulation ML : Entrée X → modèle f_θ → carte de probabilités → seuillage → masque binaire.

Pourquoi l'accuracy seule ne suffit pas. Si 98 % des pixels sont du fond, un modèle qui prédit toujours « pas de contrail » afficherait une accuracy très élevée tout en étant inutile. Il faut privilégier des métriques de recouvrement et de détection de la classe positive :

Métrique	Ce qu'elle mesure	Utilité
Dice	Recouvrement prédiction / vérité terrain	Métrique centrale, utilisée par Kaggle
IoU / Jaccard	Intersection / union	Qualité spatiale du masque
Precision	Parmi les pixels prédits positifs, combien sont corrects	Contrôle des faux positifs
Recall	Parmi les vrais pixels contrail, combien sont trouvés	Contrôle des contrails manquées

La compétition Kaggle utilise le **Dice global** comme métrique officielle.

4. La baseline U-Net

Baseline 1 — U-Net. Architecture encodeur-décodeur très utilisée en segmentation. L'encodeur compresse progressivement l'information pour apprendre des caractéristiques ; le décodeur reconstruit une carte spatiale. Les *skip connections* transmettent des détails fins de l'encodeur au décodeur. C'est la baseline recommandée pour démarrer.

Baseline 2 — DeepLabV3+. L'article de référence OpenContrails utilise une architecture basée sur ResNet et DeepLabV3+, qui exploite des convolutions dilatées et un mécanisme de décodage pour conserver plusieurs échelles de contexte.

Architectures modernes (à considérer plus tard, sans en faire un catalogue). SegFormer, OneFormer, CoaT, MaxViT ou d'autres backbones — une comparaison contrôlée de 2 ou 3 familles est préférable à une exploration exhaustive.

Modèles temporels. Le contexte temporel peut être exploité de plusieurs façons : empiler plusieurs frames comme canaux, approche 2.5D, ConvLSTM/LSTM, mélange convolutionnel temporel, ou Transformer temporel. Des solutions de haut niveau de la compétition Kaggle ont déjà exploré LSTM, Transformer et temporal mixing (voir section 5).

Loss functions. Commencer par le **BCE** (Binary Cross-Entropy) et/ou une combinaison **BCE + Dice Loss**. La Focal Loss peut être testée si le déséquilibre de classes devient problématique.

5. Ce qui existe déjà (littérature et Kaggle)

L'article de référence propose déjà un modèle utilisant le contexte temporel. Une simple comparaison « single-frame vs multi-frame » est donc une excellente expérience de reproduction, mais **pas une contribution originale suffisante à elle seule**.

La compétition Kaggle 2023 apporte des informations importantes sur l'état de l'art pratique : - La solution classée 2^e a utilisé des séquences temporelles, plusieurs mécanismes de mélange temporel et des **soft labels** obtenus à partir de la moyenne des annotateurs → [2nd place solution — temporal mixing + soft labels](#) - La solution classée 6^e a utilisé une approche 2.5D avec soft labels → [6th place solution — 2.5D + soft labels](#) - La solution classée 8^e a utilisé OneFormer avec soft/pseudo labels → [8th place solution — OneFormer + soft/pseudo labels](#) - La solution classée 30^e documente aussi ses expériences et ses échecs, ce qui est utile pour éviter de répéter les mêmes essais → [30th place solution — expériences et échecs documentés](#)

Conséquence pour notre originalité. Plusieurs solutions top-Kaggle utilisent déjà les désaccords entre annotateurs sous forme de *soft labels*. Notre piste « apprendre à partir du désaccord humain » devra donc être formulée plus finement que ce simple remplacement du masque binaire par une moyenne — voir section 6.

Autres ressources pratiques : - [Mini sample OpenContrails sur Kaggle](#) - [Projet communautaire contrails_vision sur GitHub](#)

6. Pistes de recherche à discuter

Aucune de ces pistes n'est encore figée — l'idée est d'en retenir une (ou une combinaison ciblée) après la baseline et une revue de littérature plus large :

- **Calibration de l'incertitude** : le modèle sait-il quand il est incertain, et cette incertitude correspond-elle au désaccord humain ?
 - **Learning from disagreement** : prédire à la fois la segmentation et une carte de désaccord / confiance (à différencier des simples soft labels déjà utilisés par plusieurs solutions Kaggle).
 - **Error analysis structurée** : distinguer les performances selon le type de scène, densité de cirrus, longueur ou contraste, ou âge apparent de la contrail.
 - **Architecture légère** : conserver une bonne performance avec un coût d'inférence plus faible, pertinent pour un système opérationnel.
 - **Robustesse temporelle** : mesurer quelles frames avant/après apportent réellement de l'information et si toutes sont nécessaires.
-

7. Plan de travail en 7 phases

Phase	Durée indicative	Livrable attendu
0 — Cadrage	1-2 jours	Lire le papier OpenContrails, comprendre les données, créer le dépôt GitHub, écrire la research question provisoire
1 — Data exploration	2-4 jours	Charger quelques records, vérifier dimensions, visualiser bandes et masques, produire une représentation ASH/RGB, inspecter les annotations individuelles
2 — Baseline	4-7 jours	Entraîner un U-Net simple sur la frame cible ; définir split, seed, métriques, logging et sauvegarde des modèles
3 — Reproduction temporelle	4-7 jours	Comparer single-frame à une méthode multi-frame simple et mesurer l'apport du temps
4 — Contribution	1-3 semaines	Choisir UNE piste : uncertainty-aware, ablation temporelle, architecture moderne/légère, ou analyse ciblée des cas difficiles
5 — Error analysis	3-5 jours	Visualiser TP/FP/FN, sélectionner des exemples difficiles, étudier les causes d'erreur et la calibration
6 — Finalisation	3-5 jours	README, figures, tableau de résultats, rapport court, notebook de démonstration, reproductibilité

Critère de réussite du premier jalon (fin Phase 1/2) : être capable de charger un record, visualiser correctement l'information satellite et son masque, entraîner une petite baseline et calculer un Dice sur un jeu de validation.

8. Décisions à prendre ensemble

Répartition du travail envisagée (à valider/ajuster à deux) :

Bloc	Personne A	Personne B	En commun
Littérature	OpenContrails + données	Modèles temporels / segmentation	Synthèse du gap
Données	Loader + visualisation	Preprocessing + augmentation	Validation visuelle
Modèles	Baseline U-Net	Extension choisie	Protocole commun
Analyse	Métriques	Error analysis	Interprétation
Livrable	README / figures	Code cleanup	Rapport + démo

Décisions concrètes à trancher ensemble, idéalement lors d'une première séance de travail (≈1h) : 1. Reformuler ensemble le problème et le résultat attendu (10 min). 2. Comparer nos notes respectives sur l'article OpenContrails (15 min). 3. Ouvrir un petit échantillon du dataset et vérifier les fichiers ensemble (15 min). 4. Choisir la baseline exacte à implémenter en premier — U-Net recommandé (10 min). 5. Fixer les métriques de suivi — proposition : Dice + IoU + Precision/Recall (10 min). 6. Décider du premier split (train/val), de l'environnement GPU à utiliser et de la méthode de suivi des expériences — carnet d'expériences, config versionnée, ou outil type Weights & Biases (15 min). 7. Répartir les tâches de la semaine selon (ou en ajustant) le tableau ci-dessus (10 min). 8. Fixer la prochaine réunion et le livrable attendu de chacun (5 min).

Question à ne pas trancher trop tôt : « *Quelle sera notre contribution originale finale ?* » — la bonne réponse pour l'instant est d'avoir 2-3 hypothèses candidates (section 6) et de choisir après avoir vu les premiers résultats de la baseline et fait une revue de littérature un peu plus large.

Règle de travail proposée : chaque expérience devrait avoir un objectif, une hypothèse, une configuration enregistrée, une métrique et une conclusion — éviter de lancer des modèles « pour voir » sans carnet expérimental.

Notes de prudence scientifique à garder en tête : - Les résultats Kaggle ne sont pas directement comparables aux nôtres si notre split, preprocessing ou taille d'image diffèrent. - Une amélioration de score n'est pas automatiquement une contribution scientifique — il faut expliquer *pourquoi* la méthode devrait fonctionner et analyser les résultats. - Documenter aussi les expériences qui échouent : ça évite de répéter les mêmes essais et renforce la qualité de l'analyse.

Liens et ressources — récapitulatif

Références officielles - [Article : Contrail Detection on GOES-16 ABI with the OpenContrails Dataset](#) - [Google Project Contrails](#) - [Google Research — Public Datasets](#) - [Kaggle — Compétition \(overview\)](#) - [Kaggle — Description des données](#)

Solutions Kaggle utiles pour l'état de l'art pratique - [2nd place — temporal mixing + soft labels](#) - [6th place — 2.5D + soft labels](#) - [8th place — OneFormer + soft/pseudo labels](#) - [30th place — expériences et échecs documentés](#)

Ressources pratiques - [Mini sample OpenContrails sur Kaggle](#) - [Projet communautaire contrails_vision sur GitHub](#)

Conseil : lire d'abord les sources officielles, puis les solutions Kaggle. Les write-ups de compétition sont très utiles techniquement, mais ne remplacent pas une revue de littérature scientifique.